

WEEK 09

Agent Skills 开放标准

把团队多年的「工程工艺」打包成可版本、可复用、可治理的资产

01 Why

为什么要 Skill 化

02 Pack

一目录 + SKILL.md

03 Load

渐进加载不爆 context

04 Author

写给 Agent 的指令

05 Govern

版本 + 评测 + Registry

本周划重点

本周目标 → 把团队的工艺封装成 Skill Pack，为 Week 10 的「受控 Agent」备好行动手册。Skill 是 Agent 的手册，没有标准 Skill 的 Agent 走不远。

LESSON 01 · WHY AGENT SKILLS

1. 为什么要 Skill 化：从「工程口口相传」到「可移植交付物」

工艺不是写在 Confluence 里，是封进一个能跑的包

真实场景 · 我带数据团队踩过的

前年组里做得最好的「补数 SOP」，全靠一个老工程师。他离职两个月，新人翻遍 Confluence——文档停在半年前，脚本散在三个仓库，关键那句「跑之前先锁分区」只在他脑子里。结果补错一张维表，下游 6 个看板 GMV 对不上，排查了一整天。那天我想通：团队最值钱的不是代码，是这些没人沉淀的「工艺」——而它们一直在随人走。

咱们这节聊 4 件事：

- 为什么「流程靠 Confluence、规范靠老 X」在 LLM 时代彻底走不通

- Skill 跟 Tool / Prompt / Workflow / MCP 的本质区别

- 一个 Skill 到底长什么样——简单到出乎意料

- 7 个信号，判断你的团队是不是到了 Skill 化的拐点

THE THIRD LEAP

Skill as Pack 是 AI 工程化的第三次跃迁

和 *Data as Code*、*Prompt as Code* 同代际

过去几年，数据 + AI 工程已经跃迁了两次：

1

Data as Code

数据像代码一样版本化 · 2020-2023 已普及

2

Prompt as Code

Prompt 像代码一样工程化 · 2023-2024 正普及

3

Skill as Pack

工艺打包成可移植对象 · 2025+ 刚起步

Skill = 软件工程方法 × 知识管理 的融合形态。把流程、规范、模板、脚本这些「非代码软件资产」，第一次像代码一样工程化。

KNOWLEDGE FRAGMENTATION

没有 Skill 标准之前——团队知识散在 7 个地方

它们合起来是团队的「非代码资产」，却没一个被工程化对待

知识载体	常见痛点	真实例子
Confluence / Wiki	过期最快、无版本、没人维护	「补数 SOP」半年没更新
代码注释	只服务这个项目，跨项目难复用	ETL 脚本里那句「小心 X 字段」
Prompt 模板	工具绑定，OpenAI / Claude 各一份	一份 prompt 跨工具不能用
口口相传	人在流程在，人走流程消失	「找老 X 就行」文化
Slack / 群历史	搜不到、上下文全丢	半年前的关键决策再也找不到
内部脚手架	自研框架黑盒，迁移成本高	「我们自己的 Agent 框架」

Skill 标准的目标，就是把这 6-7 类散落资产收拢成「一类对象」——一个目录、一份 SKILL.md，能被 Agent 发现、被 Git 治理、跨工具搬走。

THREE LAYERS

AI 工程化的三层进化——Skill 是最上面那层

每一层都对应这门课的某几周

L1 · Data as Code

- 数据像代码一样版本化
- lakeFS / Iceberg V3 / Dagster
- Week 2 / 6 / 14
- 2020-2023 · 已普及

L2 · Prompt as Code

- Prompt 像代码一样工程化
- Jinja2 + Git + 评测
- Week 8 Lesson 4
- 2023-2024 · 正普及

L3 · Skill as Pack

- 工艺打包可移植
- Anthropic Agent Skills
- Week 9 · 这一周
- 2025+ · 刚起步

三层是叠加不是替代 → 没有 Data as Code 的版本化，Skill 里的脚本照样不可复现。Skill 站在前两层的肩膀上。

SKILL VS OTHERS

Skill 跟 Tool / Prompt / Workflow / MCP 到底差在哪

Skill 不替代它们——是把它们组合起来的「打包格式」

对象	解决什么	跟 Skill 的关系
Tool / Function	让 LLM 调一个具体函数	Skill 内部可以调它
Prompt 模板	约束单次输出	Skill 可以把它内嵌
Workflow	编排端到端流程	Skill 可封装，但不绑工具
MCP Server	连接外部系统 / 工具 / 数据	互补：MCP 管「连接」，Skill 管「怎么用好」
Skill Pack	把工艺打包成可移植对象	把上面这些组合起来的容器

2026 架构共识 → MCP 是「神经系统」（接触世界），Skill 是「心智剧本」（接触后怎么干漂亮），subagent 是「隔离的专精脑」，三者叠加才是最强解。Skill 里用 Server:tool 调 MCP 工具 · 成本铁律：能用一个 Markdown 解决，就别上一个部署的服务。

WHAT A SKILL LOOKS LIKE

一个 Skill 长什么样——简单到出乎意料

没有 SDK、没有运行时，就是文件夹 + Markdown

```
skills/data-contract-lint/
├── SKILL.md      ← 核心：YAML 头 + 指令正文（必）
├── scripts/
│   ├── lint.py   ← 可执行检查
│   └── fix.py    ← 自动修复
├── references/
│   └── odcs-spec.md ← 按需查的资料
└── assets/
    └── template.json ← 产物模板
```

```
# ---- SKILL.md ----
```

```
---
```

```
name: data-contract-lint
description: Validate a data contract YAML
  against ODCS v3.1. Use when user mentions
  "contract" / "schema validation".
---
```

```
# 数据契约检查
```

```
当用户要验证契约时：
```

1. 用 scripts/lint.py 解析 contract
2. 对照 references/odcs-spec.md 查 6 个维度
3. 按 severity 输出 violations

老司机解读

- 本质就这些：1 个目录 + 1 份 SKILL.md（必）+ 3 个可选子目录
- 没特殊语法、没专用 SDK、没运行时依赖——就是文件 + Markdown
- YAML 头是给 Agent 看的元数据：决定「什么时候用它」
- 正文给「人 + Agent」一起读：工程师能 review，LLM 能执行
- scripts/ 把「知识」变成「可执行」——这就是工艺打包

AGENT SKILLS 2025-2026

Agent Skills 不是概念——已经是 GA 的开放标准

别再说「在观望」，标准已经收敛

时间	事件	工程含义
2025.10.16	Anthropic 首发 Agent Skills (beta)	Claude / Claude Code 原生支持目录发现 + 渐进加载
2025.12.18	开源为开放标准 agentskills.io	不再绑 Anthropic · MCP 同期捐 Linux 基金会
2026.02	企业管控 + 合作伙伴技能目录上线	Atlassian / Figma / Canva / Notion / Stripe / Zapier
2026 上半年	Codex / Cursor / Gemini CLI / Copilot 等 30+ 跟进	SKILL.md 成跨厂商事实标准 · 仓库约 15 万 star

选择不再是「要不要 Skill 化」→ 到 2026 年 3 月，OpenAI Codex、Cursor、Gemini CLI、Copilot、JetBrains、Block Goose 等 30+ Agent 已跑通 SKILL.md。它正在成为「Agent 界的 REST」，标准已收敛，自己再造一套就是纯工程债。



WHEN YOU NEED SKILLS

什么时候该开始 Skill 化——7 个信号自测

中 3 条以上，就别拖了

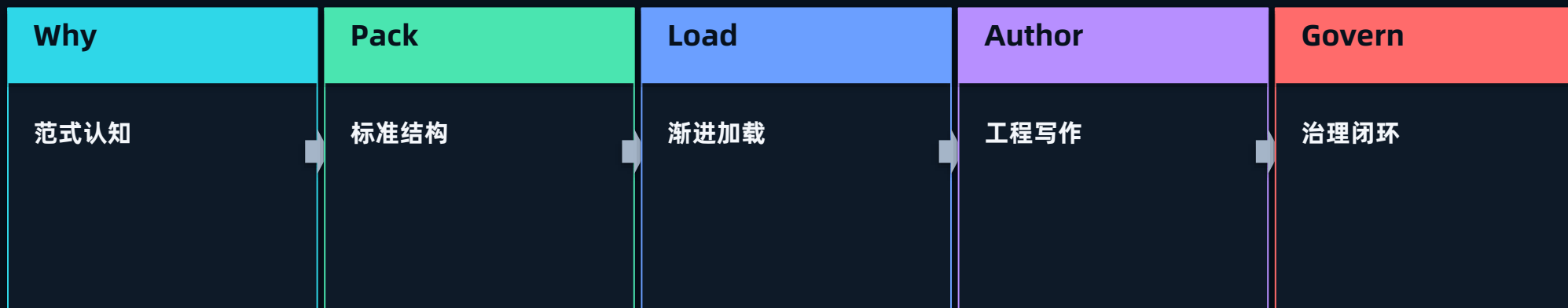
信号	具体表现	严重度
流程靠「找老 X」	关键操作没有可执行文档	高
同一规范多份副本	不同项目各写一份契约检查	高
跨工具迁移痛	OpenAI / Claude / Cursor 各做一遍	高
Runbook 不可执行	Markdown 写着，但没人照它做	中
新人 onboarding 慢	没有「照这个做」的标准包	中
合规要求出现	审计要看「流程文档 + 执行证据」	高 · 必须做

Skill 化不是「高级技巧」，是团队规模化的必要基建 → 5 个人的小队可以靠脑子，30 个人 + 多项目 + 要过审计时，没有 Skill 标准必乱。

WEEK 09 MAP

Week 9 五节课的内在逻辑——从认知到治理

这条链最终接到 Week 10 的「受控 Agent」



Skill 是 Agent 的「行动手册」，Agent 是执行者——Week 10 起两者合体才是「能办事的 AI」。没有 Skill 标准的 Agent，必然走不远。

LESSON 02 · SKILL PACK ANATOMY

2. Skill Pack 标准结构：一个目录 + SKILL.md + 三件配套

结构不是我拍脑袋的——是 Anthropic 标准里规定的

真实场景 · 90% 团队第一次写 Skill 都这样

我见过太多团队第一次做 Skill——代码、文档、模板、Prompt 全塞进一个文件夹，SKILL.md 写成两千行大杂烩。换个工具就跑不起来，想改一行得通读全文。Skill 难看不是因为复杂，是因为没人告诉他们：这东西有标准结构，scripts 放动词、references 放名词、assets 放形容词，该放哪放哪，简单到不可思议。

咱们这节聊 4 件事：

- SKILL.md 的两部分：YAML 头（给机器）+ 正文（给人和 Agent 一起读）

- frontmatter 到底哪些字段是「官方必填」，哪些是团队自己加的

- scripts / references / assets 三个目录的边界怎么划

- 一段生产级 lint.py 长什么样——4 个工程要点

JUST A FOLDER

Skill 的本质是「一个目录」——但每个组件都有明确职责

不需要框架、SDK、特殊运行时，什么都不需要

SKILL.md

给 LLM 的入口：元数据 + 指令

scripts/

给系统执行的可运行物

references/

给 LLM 按需检索的资料库

assets/

给产物提供的模板

简单结构承载复杂能力，不绑任何运行时——这是 Skill 能「跨工具搬走」的根。结构越简单，越可移植。

TWO PARTS

SKILL.md 由两部分组成——一份给机器，一份给人

YAML 头决定「何时用」，正文决定「怎么做」

YAML Frontmatter

- name: 唯一标识
- description: Agent 发现依据
- 机器读 · 决定何时激活
- 官方只强制这两个字段

Markdown 正文

- 触发 / 反触发条件
- 执行步骤 + 约束 + 禁区
- 正反例
- 人 + Agent 双观众读

双观众阅读

- 工程师 review 看正文
- Agent 加载看两部分
- 改 = 改 .md 走 PR
- Git 跟踪 = 天然版本化

「改 .md 走 PR」这一条，就是 Skill 工程化的命门 → 工艺第一次能像代码一样被 diff、被 review、被回滚。

FRONTMATTER SPEC

YAML 头：官方只强制 2 个字段，别被一堆字段吓到

把团队扩展字段和 spec 必填字段分清楚

```
---
# === 官方 spec 强制 (缺一不可) ===
name: data-contract-lint    # ≤64 字符
                             # 小写/数字/连字符
                             # 禁用 anthropic、claude
description: |               # ≤1024 字符 · 第三人称
  Validate a data contract YAML against
  ODCS v3.1. Use when user mentions
  "contract" / "schema validation".
  必须同时写清「做什么」+「何时触发」

# === 以下都是团队工程化扩展 (非必填) ===
version: 1.2.0              # SemVer · 治理用
inputs:
  - contract_path: string
outputs:
  - violations: array
  - severity: enum[info,warn,error]
requires: [audit-log-writer >=1.0.0]
permissions: { filesystem: read-only }
---
```

老司机解读 · 准确性

- 官方 spec 只强制 name + description——这点很多教程都写错
- name ≤ 64 字符，小写/数字/连字符，禁用 anthropic、claude 保留词
- description ≤ 1024 字符，第三人称，必须写清「做什么 + 何时用」
- description 是 Agent 发现的唯一依据：写不好就找不到，写太宽就误激活
- spec 可选字段只有 license / compatibility / metadata / allowed-tools (实验) · version / inputs 是团队自加

THREE COMPANION DIRS

scripts / references / assets——三者边界别混

一句口诀：动词 / 名词 / 形容词

目录	装什么	谁读它	何时加载
scripts/	可执行代码（.py / .sh）	系统运行时执行	执行时按需调用
references/	规范、案例、资料（.md）	LLM 检索补上下文	LLM 决定要不要读
assets/	模板、初始化文件	Skill 生成产物时用	生成阶段加载

最容易混的是 references 和 assets → references 是「给 LLM 查的资料」（如 ODCS spec、正反例），assets 是「Skill 生成产物的模板」（如空白契约 yaml）。一个是输入侧、一个是输出侧。很多团队全堆一起，结果 Skill 既不可移植又不可治理。

SCRIPTS DESIGN

scripts/ 的 4 个设计要点——不是「丢几个脚本」

scripts 是 *Skill* 里工程化要求最高的部分

1 · 幂等可重入

- 同样输入跑两次结果一致
- 补数 / 重试场景必须如此
- Week 6「3 层幂等」同样适用

2 · 结构化输出

- 输出 JSON, 不要 print
- LLM / 下游能直接 parse
- argparse + json.dumps 起步

3 · 明确错误码

- 0 成功 / 1 输入错 / 2 规则违反
- 让上游决定 重试 / 报错 / 升级
- 区分「能不能重试」

4 · 零外部依赖

- 能用标准库就用标准库
- 必须用三方 → requirements 显式声明
- 依赖多 = 跨环境跑不起来

判 **scripts** 好不好就一条：把它从 **Skill** 里抠出来，能不能在一台干净的机器上一行命令跑通。跑不通就不算工程化。

REAL SCRIPT

OmniSupport 项目里 data-contract-lint 的 scripts/lint.py

生产级 Skill script 该长的样子

```
#!/usr/bin/env python3
"""scripts/lint.py - 校验 data contract YAML
Exit: 0 全过 / 1 输入错 / 2 校验失败"""
import sys, json, yaml, jsonschema, argparse
from pathlib import Path

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--contract-path", required=True)
    args = p.parse_args()
    f = Path(args.contract_path)
    if not f.exists():
        print(json.dumps({"error": "file_not_found"}))
        sys.exit(1)
    contract = yaml.safe_load(f.read_text())
    violations = validate(contract)
    print(json.dumps({"violations": violations,
        "severity": "error" if violations else "ok"}))
    sys.exit(2 if violations else 0)

if __name__ == "__main__":
    main()
```

老司机解读

- docstring 写明 exit code · Agent 才知道怎么解读结果
- argparse 参数与 SKILL.md 的 inputs 严格对应
- 输出全部 json.dumps · Agent 直接 parse，不靠猜
- exit code 区分「输入错」和「内容错」· 决定能不能重试
- 只用标准库 + yaml / jsonschema · 搬到任何 Python 环境都能跑

REFERENCES & ASSETS

references 和 assets——别混在一起

references 给 LLM 查, assets 给产物当模板

references/ 是按需检索包

- 放规范、术语表、正反例
- LLM 执行时按需读, 不是启动全加载
- 只放真会用到的 5-10 篇
- 多了反而拉低检索效率

assets/ 是产物模板

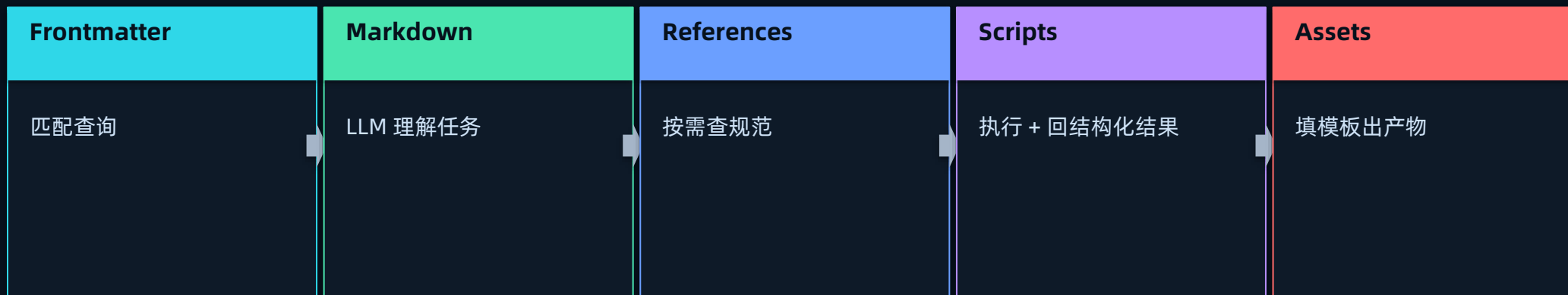
- 如「生成契约」→ 空白 template.yaml
- Skill 把变量填进模板出产物
- 也可放 logo / 配置 / prompt 段
- 「生成时用」的放这

记死这条 → 你要「读它来理解」的, 放 references; 你要「拿它来生成」的, 放 assets。输入侧 vs 输出侧, 别让 SKILL.md 自己膨胀。

HOW A SKILL EXECUTES

Skill 执行时，五个组件如何协作

各司其职、按需组合——放错位置就不可治理



这条链是 Skill 设计的核心 → 发现靠 frontmatter、执行靠正文、补上下文靠 references、干活靠 scripts、出产物靠 assets。任何一个职责放错位置，整个 Skill 就变得不可治理。

LESSON 03 · PROGRESSIVE DISCLOSURE

3. 渐进加载：Skill 规模化时不爆 context、不误激活

Skill Spec 里设计最精巧的一环

真实场景 · 一笔把 context 烧爆的账

我算过一笔账：团队 Skill 做到 50 个、每个 SKILL.md 两千 token，要是 Agent 一启动就全量加载——光元数据吃掉 10 万 token。按 Claude 输入约 \$3/百万 token 算，每次对话还没开口就先烧 \$0.3，真实对话窗口被挤得所剩无几。做到 500 个 Skill，直接 100 万 token，窗口当场爆。这不是假设，是规模化必然撞的墙。

咱们这节聊 4 件事：

- 三段式加载：Discovery / Activation / Execution，每段加载量级差几十倍

- 每段的 token 预算——这是设计 Skill 系统的硬约束

- description 怎么写，决定 Skill 能不能被 Agent 找到

- Skill Routing 三种策略，按团队规模选哪个

LOAD ON DEMAND

Skill 数量增长是必然——所以加载必须「按需」

否则 *context* 会被规模杀死

一个团队的 Skill 从 5 → 50 → 500，是 Skill 化成功的标志。但如果「启动就全读」：

50 个 Skill × 2K token

= 10 万 token 元数据

窗口紧张

500 个 Skill × 2K token

= 100 万 token

直接爆窗口

Progressive Disclosure 把加载拆成三段——元信息 / 指令 / 资源，每段按需触发。这是让 Skill 系统能横向扩展的核心设计，也是规模化的唯一出路。

THREE LOAD STAGES

渐进加载三段式——每段加载量级差几十倍

对齐 Anthropic 官方的三级模型

Stage 1 · Discovery

- 只加载 name + description
- 约 100 token / skill
- 启动时常驻 system prompt
- 几百个 Skill 也不爆

Stage 2 · Activation

- 命中才读整篇 SKILL.md
- 通常 < 5K token
- 官方建议正文 < 500 行
- 决定 Agent 怎么执行

Stage 3 · Execution

- 按需读 references / 跑 scripts
- 脚本只回结果，不进 context
- 没用到的文件 = 0 token
- 决定产物长什么样

官方关键数字 → L1 每个 Skill 约 100 token 常驻、L2 整篇通常 <5K、SKILL.md 建议 <500 行。脚本是「执行」不是「加载」，代码不进窗口，只有输出进。

TOKEN BUDGET

三段式的 token 预算——设计 Skill 系统的硬约束

渐进加载把百级 Skill 的元数据成本固定在 5-10K

阶段	每个 Skill	触发时机	50 个 Skill 总量
Stage 1 Discovery	约 100 token	启动时	约 5K（可控）
Stage 2 Activation	1-2K token	命中后	约 5K（按需）
Stage 3 Execution	取决于 ref + 脚本输出	执行时	动态
全量加载（错误做法）	3K-10K token	启动时	50K-500K（爆）

记住这条曲线 → 全量加载是「线性爆炸」，渐进加载是「常数级」。Skill 从 50 涨到 500，启动成本几乎不变——每次对话的 Skill 元数据成本从约 \$0.30 压到 \$0.015。

GOOD VS BAD DESCRIPTION

好的 description vs 坏的——决定 Skill 能否被发现

description ≤ 1024 字符, 第三人称, 写清做什么 + 何时用

description	问题	后果
"Lint data contract"	太短 / 缺触发关键词	Agent 不知道何时用
"helps with various data quality tasks..."	太泛 / 含糊	所有相关问题都误激活
"Validate ODCS contract YAML. Use when user mentions contract / schema validation."	明确 + 触发词 + 边界	Agent 精准路由 ✓
"Internal tool for support team"	面向人写, 不是给 Agent	Agent 完全理解不了

description 公式 → 「动词短语 + 输入对象 + Use when X / Y / Z」。它是 Stage 1 唯一能看到的東西，写好写坏直接决定 Agent 找不找得到这个 Skill。这是整个渐进加载里信息密度最高的一句，值得反复打磨。



DESCRIPTION RULES

写好 description 的 5 条规则

从真实生产里路由失败的案例反推出来的

1 · 含具体触发关键词

Use when user mentions X / Y / Z · 把用户可能说的词列上

2 · 动词具体

别写 helps with, 要写 Validate / Generate / Compute

3 · 边界清晰

别写 various, 写明 for X only · 划出反例

4 · 长度适中

50-150 token 最佳 · 太短没信息, 太长信噪比降

5 · 带一两个反例

Not for ... · 反例比正例更能减少误激活

反例比正例更值钱 → 「Not for X」一句, 能挡掉一大片误激活。



MINIMAL LOADER

渐进加载的最小实现——自研 Agent 里也能用

这套模式能嵌进任何 Agent 框架

```
class SkillRegistry:
    def __init__(self, root: Path):
        self.metas = self._discover(root)

    # Stage 1: 启动只读 frontmatter
    def _discover(self, root):
        metas = []
        for md in root.rglob("SKILL.md"):
            fm = read_frontmatter(md) # 只解析头
            metas.append(SkillMeta(
                name=fm["name"],
                description=fm["description"],
                path=md.parent))
        return metas

    # Stage 2: 命中才读整篇
    def activate(self, name):
        m = self._find(name)
        return (m.path / "SKILL.md").read_text()

    # Stage 3: LLM 决定调脚本 / 读 ref
    def run_script(self, name, script, **kw):
        m = self._find(name)
        return execute(m.path/"scripts"/script, **kw)
```

老司机解读

- `_discover()` 只读 frontmatter · 1000 个 Skill 也只产生百级 K token
- `activate(name)` 命中才加载整篇 · 用到才读
- `run_script` / `read_reference` 是 Stage 3 的两类按需操作
- 脚本走 subprocess · 输出回 LLM, 代码不进 context
- 这套模式 LangChain / LangGraph / LlamaIndex / CrewAI 都能套

SKILL ROUTING

Skill Routing 的 3 种策略——按规模选

本质是个轻量「检索 + LLM 决策」问题

策略	怎么做	优点	局限
LLM 直接选	把所有 description 给 LLM 让它选	准确率高	token 贵 · 不适合 100+
Embedding 检索	对 description 做向量检索	低成本 · 可规模化	要维护 vector index
规则 + LLM 混合	关键词预过滤 top-10 再 LLM 选	准确 + 高效	要维护关键词字典

2025 选型口诀 → 小规模 (<30 Skill) 用 LLM 直接选，简单够用；大规模 (≥50) 上 Embedding + LLM 二阶段。这跟 Week 8 的 RAG 检索范式同构——其实 Skill 发现就是一个「检索问题」，你 RAG 那套经验直接能搬过来。



ANTI-PATTERNS

渐进加载的 5 个常见反模式

Skill ≤ 30 时不明显，超过 50 必须做对

反模式	表现	正确做法
启动全加载	所有 SKILL.md 整篇读进 context	只读 frontmatter
description 写成内部文档	面向人写不是给 Agent	面向 Agent 写 + 触发词
references 堆进正文	Activation 阶段就爆	挪到 references/ 按需读
Skill 之间硬依赖	调 A 必须先调 B	保持独立 + 显式 requires
50+ 还在 LLM 直选	token 成本爆炸	上 embedding routing

踩坑提醒

我见过一个团队 Skill 做到 60 多个还在启动全量加载。每次对话固定先烧十几万 token，月底账单翻倍才发现。根因不是技术难，是没人把「只读 frontmatter」这一条立成规矩。规模化的坑，往往是工程纪律问题，不是技术问题。

LESSON 04 · AUTHORIZING A SKILL

4. 写好 SKILL.md：从「描述什么是 X」到「告诉 Agent 如何做 X」

这是写作范式的切换，不是格式转换

真实场景 · 抄 Wiki 写 Skill 的翻车现场

有个团队把 Confluence 上一篇写得很漂亮的 SOP，加个 YAML 头就当 Skill 提交了。上线后 Agent 三天两头用错——用户问数据质量，它激活了 RAG 检查；该拦的越界操作全放过去。根因就一个：Confluence 是写给「人看完自己判断」的，SKILL.md 是写给「Agent 看完立刻执行」的。这两种文档的写法，是反的。

咱们这节聊 4 件事：

- 给人看的文档 vs 给 Agent 看的指令——写作风格完全相反

- 生产级 SKILL.md 的 7 个必要章节

- Trigger + Not-Trigger 都要写——避免误激活最有效的手段

- 4 类约束 + 5 类失败模式，必须显式声明

INSTRUCTION, NOT DOC

SKILL.md 不是「使用文档」——是写给 Agent 的可执行指令

两种文档的写作哲学是相反的

人类文档 (Wiki)

- 背景多 / 解释多
- 步骤可省, 留人判断
- 例子可选
- 边界含糊: 通常不要...
- 目标: 让人理解

Agent 指令 (SKILL.md)

- 背景少 / 直奔执行
- 步骤明确, 每步给动作
- 正反例必备
- 边界严格: 禁止 X / Y / Z
- 目标: 让 Agent 不出错

记死 → SKILL.md 不能直接抄 Wiki。必须按 Agent 的「阅读方式」重新组织: 先触发条件, 再步骤, 再禁区, 最后正反例。

HUMAN VS AGENT

给人看 vs 给 Agent 看——逐维度对照

把这张表贴墙上，写之前过一遍

维度	人类文档（Wiki）	Agent 指令（SKILL.md）
第一段	背景 / 历史 / 为什么	触发条件 / 何时使用
步骤	可省略，留人判断	每一步给具体动作
例子	可选	必备——正例 + 反例
边界	含糊（通常不要...）	严格（禁止 X / Y / Z）
错误处理	出错再说	提前声明 failure mode
输出格式	自由描述	强制 JSON / 结构化

这张表是写好 SKILL.md 的第一步 → 不是把 Wiki 拷过来加个 YAML 头，是按右边这列从头重写。两种文档，两套大脑。

SEVEN SECTIONS

生产级 SKILL.md 的 7 个必要章节

分三组：发现 / 执行 / 证据

Identity · 发现

- name / version
- description
- triggers / not-triggers
- inputs / outputs
- Agent 用来发现 + 路由

Procedure · 执行

- Steps 具体步骤
- Constraints 约束
- Failure Modes 失败处理
- 每步对应具体动作
- Agent 用来执行

Evidence · 证据

- Examples 正反例
- Audit Fields 审计
- Citation Format 证据格式
- Reference Mapping
- Agent 用来产可验证结果

7 段不是凑数 → 少了 not-triggers 就误激活，少了 constraints 就越界，少了 audit 就复盘不了。每一段都对应一类生产事故。

TRIGGER & NOT-TRIGGER

正反向触发都要写——避免误激活的关键

只写「何时用」不写「何时不该用」= 一定误激活

TRIGGER · 何时用

- 上线前要求「检查 RAG 输出」
- 调试「为什么这条没引用」
- CI 中跑预发回归
- 描述具体场景，不是「用于 RAG」

NOT-TRIGGER · 何时不用

- 问「模型答得准吗」→ 用 eval-set-runner
- 问「数据质量」→ 用 data-contract-lint
- 把易混的 Skill 显式点名
- 直接告诉 Agent：那种情况不是我

Not-Trigger 是 SKILL.md 区别于 Wiki 的本质特征 → Wiki 从不会写「我不管什么」，但 Agent 指令必须写。把容易抢活的兄弟 Skill 直接点名，路由准确率立刻上一个台阶。

REAL SKILL.MD

OmniSupport 项目 · rag-contract-check 的完整 SKILL.md

看一份生产级 Skill 指令长什么样

```
---
name: rag-contract-check
description: |
  Validate a RAG API response against the
  OmniSupport contract (evidences>=1, structured,
  audit fields). Use when reviewing RAG output /
  pre-deploy check. Not for: data quality, eval.
---
# RAG 响应契约校验
## TRIGGER
- 上线前要求「检查 RAG 输出」
- 调试「为什么这条回答没引用」
## NOT-TRIGGER
- 问「模型答得准吗」→ eval-set-runner
- 问「数据质量」→ data-contract-lint
## STEPS
1. scripts/check.py --response-path={inputs}
2. 解析 violations: missing_evidence /
   schema_invalid / missing_audit
3. error 进 gate=fail
## CONSTRAINTS
- read-only, 不改原 response
- 离线运行, 不调外部 API
- 输出必须含 audit_log_id
```

老司机解读

- TRIGGER 写 3 类具体场景 · 不是一句「用于 RAG」
- NOT-TRIGGER 把易混 Skill 点名 · 直接说「那不是我」
- STEPS 每步对应具体动作 · 用 scripts/check.py、解析 violations
- CONSTRAINTS 是边界承诺 · read-only、离线、必须审计
- EXAMPLES 不写正文 · 放 references/examples, 避免 Stage 2 加载贵

CONSTRAINTS

Constraint 段的 4 类核心约束——缺一类都可能出事故

约束不是建议，是 Agent 执行时的硬边界

证据 · Evidence

- 产出必须能反向定位原文
- 对应 Week 2/7/8 evidence_anchor
- 含 source / page / version

结构 · Schema

- 输出必须符合声明 schema
- 对应 Week 8 Structured Outputs
- 不能自由发挥格式

审计 · Audit

- 每次执行写 audit log
- 含 trace_id / actor / args_hash
- 事故复盘的根

边界 · Boundary

- 不读 X、不调 Y、不动 Z
- 显式禁区
- 最有效的防线是不让进危险区

约束要写「硬」不写「软」→ 别写「建议不要删」，写「禁止删除」。Agent 对「建议」是会打折扣的，对「禁止」才当回事。

FAILURE MODES

5 类失败模式——不同失败对应不同行为

生产级 Skill 必须显式声明「什么时候会失败」

失败类型	场景	返回	是否重试
Input Invalid	输入格式错 / 文件不存在	error_code=input + 建议	否, 让 Agent 改输入
Validation Failed	业务校验没过	violations[] + severity	否, 结果就是 fail
Transient	网络抖动 / 限流	error_code=transient + retry_after	是, 3 次以内
Permanent	资源不存在 / 没权限	error_code=permanent	否, 升级
Partial Success	5 条只检了 3 条	results + remaining	让 Agent 决策

把「出错」当预期行为来设计 → Agent 最怕的不是失败, 是「不知道自己失败了」。声明清楚每类失败返回什么、能不能重试, Agent 才能执行前就知道边界, 而不是出错了瞎试。这是 Skill 从「能跑」到「生产可用」的分水岭。

WRITING ANTI-PATTERNS

SKILL.md 写作的 5 个反模式

最难改的是写作思维：从给同事写，切到给 Agent 写

反模式	后果	正确做法
抄 Wiki	Agent 找不到触发、执行不到位	按 7 段重写
只写正向步骤	误激活、越界、卡死	正反向都写
examples 放正文	Stage 2 加载贵	放 references/examples/
不写 audit	事故复盘失败	强制 audit_log_id
约束写得软	Agent 觉得可以违反	写「禁止 X」

踩坑提醒

从「给同事写文档」切到「给 Agent 写指令」，至少练 5-10 份才转得过来·笨办法：写完让一个干净 Agent 照着跑，它哪步卡住就是你哪段没写清·别硬写：Anthropic 官方 skill-creator 技能交互生成骨架 + 自动测试 + 跑 benchmark + 失败分析。

LESSON 05 · GOVERN & ECOSYSTEM

5. Skill 治理：让 Skill Pack 像 npm package 一样可信、可演进

做单个 Skill 容易，做「能演进的 Skill 系统」难

真实场景 · Skill 改一行、线上炸一片

我见过最隐蔽的一次事故：一个被很多业务共用的 Skill，有人改了一步措辞，觉得「就润色一下」，直接合了。结果三个下游 Agent 行为全变了，客服话术一夜走样，两天后才被投诉量带出来。从那以后我立了死规矩：Skill 改动必须像改代码一样走 PR + 回归 + 版本，一个都不能少。Skill 是 Agent 的行动手册，改手册等于改业务行为。

咱们这节聊 4 件事：

- Skill 治理的 3 大支柱：版本 / 评测 / Registry

- Skill PR 工作流 + GitHub Actions 实战配置

- Skill 会跑代码——信任边界和安全怎么划（原 deck 缺的一块）

- 把 Skill 绑进 Release Manifest，实现「数据 + 索引 + Prompt + Skill」四元原子绑定

SKILL IS A PRODUCT

Skill 是工程产品——必须像软件一样版本、评测、发布、回滚

Skill 不是黑箱，是 Markdown + 脚本——所以能直接用软件工程方法治理

Skill 一旦超过 5 个，这些问题就来了：

改一个 Skill 的步骤——会不会影响其他 Skill？

某个 Skill 上周还行、这周突然不灵——是 Skill 改了，还是模型升级了？

A 团队的 Skill 想给 B 团队用——版本兼容吗？

这些都是「分布式系统」问题 → 但 Skill 不是黑箱，是文本 + 脚本。所以版本化、评测、回滚、依赖管理这套软件工程方法，可以原样搬过来。一个都不能少。

THREE PILLARS

Skill 治理的 3 大支柱

版本 / 评测 / Registry

Version · 版本

- SemVer 语义化版本
- Git tag 锚定
- CHANGELOG.md 维护
- release_id 绑数据+索引+Prompt
- 回滚 = 切 release_id

Eval · 评测

- Skill golden set
- PR 自动回归 · 接 Langfuse / Ragas
- 上线前 + 上线后双跑
- failure case 沉淀进反例库
- 回归不过不能合并

Registry · 仓库

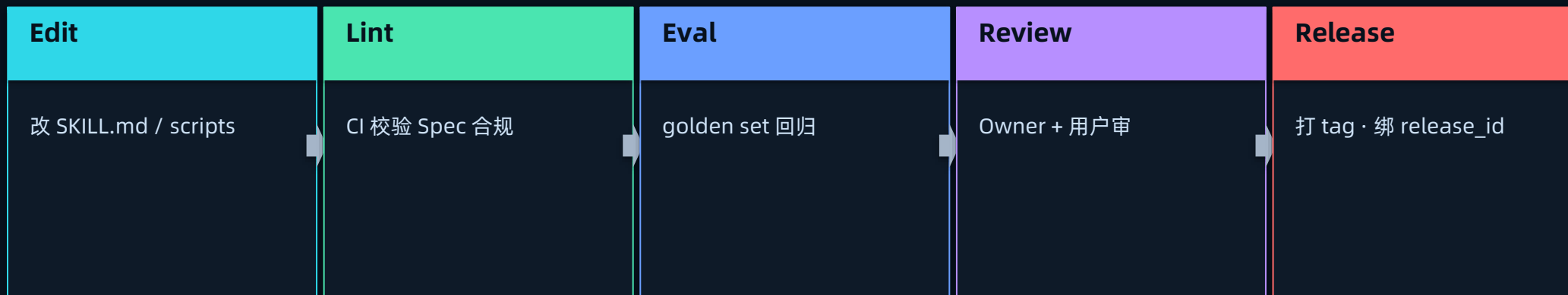
- 私有 / 公开 Skill 仓库
- discovery API
- 依赖关系 + 兼容矩阵
- 跨团队 / 跨工具复用
- 类似 npm / pypi

不评测就上线 = 拿客户流量当测试集 → golden set + PR 自动回归，是 Skill 治理里性价比最高的一件事。先把这个搭起来。

SKILL PR WORKFLOW

Skill PR workflow——5 步全自动

和代码 PR 完全同构，从此享受同样的工程治理



关键不在工具，在纪律 → 任何用 Git 的团队，今天就能把这 5 步搭起来。Skill 改动从此走 PR，不再有人「润色一下」直接合进生产。

SKILL CI

Skill 仓库的 GitHub Actions 实战配置

不是高级技巧，用 Git 的团队都能立刻搭

```
name: Skill Pack CI
on:
  pull_request:
    paths: ["skills/**"]
jobs:
  lint:
    steps:
      - run: python3 tools/skill_lint.py skills/
        # 校验 frontmatter + 正文 7 段结构
      - run: python3 tools/script_lint.py skills/
        # 校验 scripts 有 docstring + exit code
  eval:
    needs: lint
    steps:
      - run: |
          python3 tools/skill_eval.py \
            --skills $(git diff --name-only main) \
            --baseline release/last-stable.json
      - run: | # 指标退化 >2% 直接 fail
          test $(jq '.regressions' report.json
            | length) -eq 0
  publish:
    if: github.ref == 'refs/heads/main'
    needs: eval
    steps:
      - run: tools/skill_release.sh # bump + tag
```

老司机解读

- lint · 强制符合 Spec（7 段 + scripts 规范），违反不让合
- eval · 每个改动的 Skill 跑 golden set
- 指标退化 > 2% 直接 block · 这条阈值是回归防线
- publish · 合到 main 自动打 tag + 推私有 registry
- 说白了 · Skill 改动从此享受和代码一样的 CI 门禁

REGISTRY ARCHITECTURE

Skill Registry 的 3 类架构——按组织规模选

别一上来就上重型方案

架构	怎么实现	适用	维护成本
Git Submodule	skills 拉进每个项目	小团队 · <10 Skill	低
Private Git Repo	统一 skills 仓库 + 各项目引用	中型 · 10-50 Skill	中
Skill Registry API	专门服务暴露 REST API	大型 · >50 Skill	高
公开 Marketplace	类似 npm 公开仓库	社区共享	极高

2025 默认建议 → 中型团队从 Private Git Repo 起步，超过 50 个 Skill 再上 Registry API。我见过太多团队一上来就想做「企业级 Skill 中台」，结果三个月还没上线第一个 Skill。先让它跑起来，再谈治理。

REGISTRY FEATURES

Skill Registry 必备的 5 个能力

少了任何一个，就不算「真正的 Registry」

1 · Discovery API

search / list / get 返回 frontmatter · Agent 按 description 检索

2 · Version Resolution

支持 SemVer 范围 (^1.2.0) · 消费方锁兼容版本

3 · Dependency Graph

requires 解析成依赖图 · 防循环依赖、查兼容冲突

4 · Audit Log

谁拉了哪个 Skill、何时用 · 接 OpenLineage 溯源给 Week 14 治理

5 · Mirror / Cache

本地缓存防网络抖动 · 生产 Skill 调用要有 SLA

Registry 的灵魂是「依赖 + 版本」→ 没有这两样，它只是个文件夹，不是 Registry。

RELEASE MANIFEST

把 Skill Pack 绑进 Release Manifest——治理闭环

Week 8 绑了索引/Prompt/模型, 现在补上 Skill 这一元

```
# release/manifests/rag-v2026.05.18-001.yaml
apiVersion: omnisupport.rag/v2 # v2 加 skill_pack
kind: RAGRelease
metadata:
  release_id: rag-v2026.05.18-001
spec:
  index: # Week 6/7
    pgvector_snapshot: idx-20260518
  prompt: # Week 8
    rag_answer: v3.2.1
  model:
    generator: claude-sonnet-4-6
  skill_pack: # Week 9 新增
    registry: skills.internal
    version: 1.4.0
  skills:
    - { name: rag-contract-check, version: 2.1.0 }
    - { name: data-contract-lint, version: 1.2.0 }
  digest: sha256:abc123... # 整 Pack 指纹
  rollout:
    strategy: canary
    auto_rollback_on:
      - { metric: skill_failure_rate, threshold: 0.01 }
```

老司机解读

- model 段锁 Claude Sonnet 4.6 · skill_pack 锁 Skill 版本 · 四元原子绑定
- digest 是整 Pack 内容指纹 · 任何 Skill 改动都改 hash
- 事故复盘能精确定位是哪个 Skill 改了
- auto_rollback 加 skill_failure_rate · 失败率超阈自动回滚
- 这就是治理闭环 · Week 6-9 所有工程对象同一个 release_id 管

SECURITY · TRUST BOUNDARY

Skill 会跑代码——信任边界必须先划清

原 deck 缺的一块：装一个 Skill = 在你环境里跑别人的代码

致命三件套 (Lethal Trifecta) · 私有数据 + 不可信内容 + 外部传输，三者齐了就可被窃取

已不是 PoC · Antiy 「ClawHavoc」行动抓到 1,184 个恶意 Skill，Unit 42 实拍野外注入

供应链 > 提示注入 · Check Point 证实：clone 不可信项目、点同意前就能 RCE + 偷 key

只用可信来源 · 自己写的、Anthropic 官方、过审的伙伴目录；其余 Skill 逐文件审

最高危是 Skill 里 fetch 外部 URL · 能离线就 permissions 收窄；破坏操作走「计划→校验→执行」

踩坑提醒

OWASP 已立「Agentic Skills Top 10」· 一个跑代码的 Skill 权限 = 它能碰到的一切，按生产服务审、别当 Markdown 读 · 沙箱按面不同：API 无网络、Claude Code 全网络，越放越要收 permissions。

ECOSYSTEM 2026

Skill 怎么装、怎么分发——四个落地面

2026 年 Skill 已全面进入 Claude 全家桶

落地面	怎么装	注意
Claude Code	.claude/skills/（项目）或 ~/.claude/skills/（个人）	自动发现 · /skill-name 触发
Claude API	container.skills + beta header	配 code execution + Files API · 单请求 ≤ 8 个 Skill
claude.ai	设置 > Features 上传 zip	Pro / Max / Team / Enterprise 才有
分发 / 治理	anthropics/skills · /plugin · 企业版集中分发	Team/Enterprise 按组授权 + 可审计 · 各面不互通

Skill 能装进 subagent → 父 Agent 引用 Skill，派一个隔离的 subagent 去执行，正好接上 Week 10 受控 Agent。2026 企业版已支持集中分发 + 按组授权 + 可审计——Skill 治理从「个人上传」升级成「组织级管控」。

WEEK 09 · THE END

你已经有了一个可演进的 Skill 系统

从范式认知 → Pack 结构 → 渐进加载 → 工程写作 → 治理生态——团队工艺第一次有了可移植的容器

本周交付物（已 push 至 GitHub 仓库）

Skill v0.1 · data-contract-lint

[skills/data-contract-lint/](#)

Skill v0.1 · ingest-backfill-runbook

[skills/ingest-backfill-runbook/](#)

Skill v0.1 · release-check

[skills/release-check/](#)

Skill v0.1 · rag-contract-check

[skills/rag-contract-check/](#)

Skill v0.1 · prompt-release

[skills/prompt-release/](#)

Skill CI Workflow

[.github/workflows/skill-ci.yml](#)

下周 → Week 10 受控 Agent · 工具契约化 / 动作权限 / HITL 节点 / Function Calling——让 Skill 驱动 Agent, 从「答」到「办」。